

Data Structure

10. 완전탐색과 백트래킹

10주차: Exhaustive Search, Backtracking

목차 INDEX

01 완전탐색

- Brute Force
- Search Space
- Permutation
- Combination
- Subset

02 백트래킹

- Backtracking
- State Space Tree
- Pruning

03 Greedy Algorithm

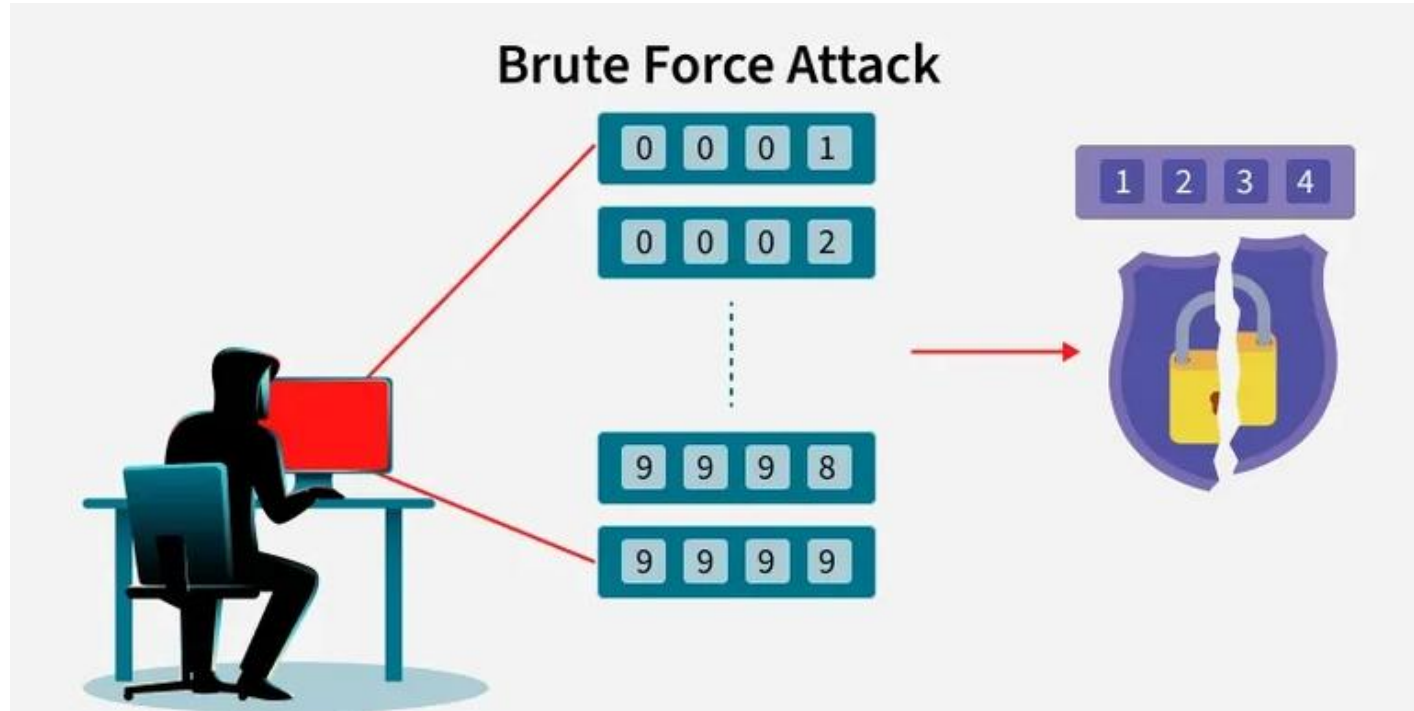
- Greedy Algorithm
- Local / Global Optimum
- Greedy의 성공과 실패 조건

01

완전탐색

Brute Force

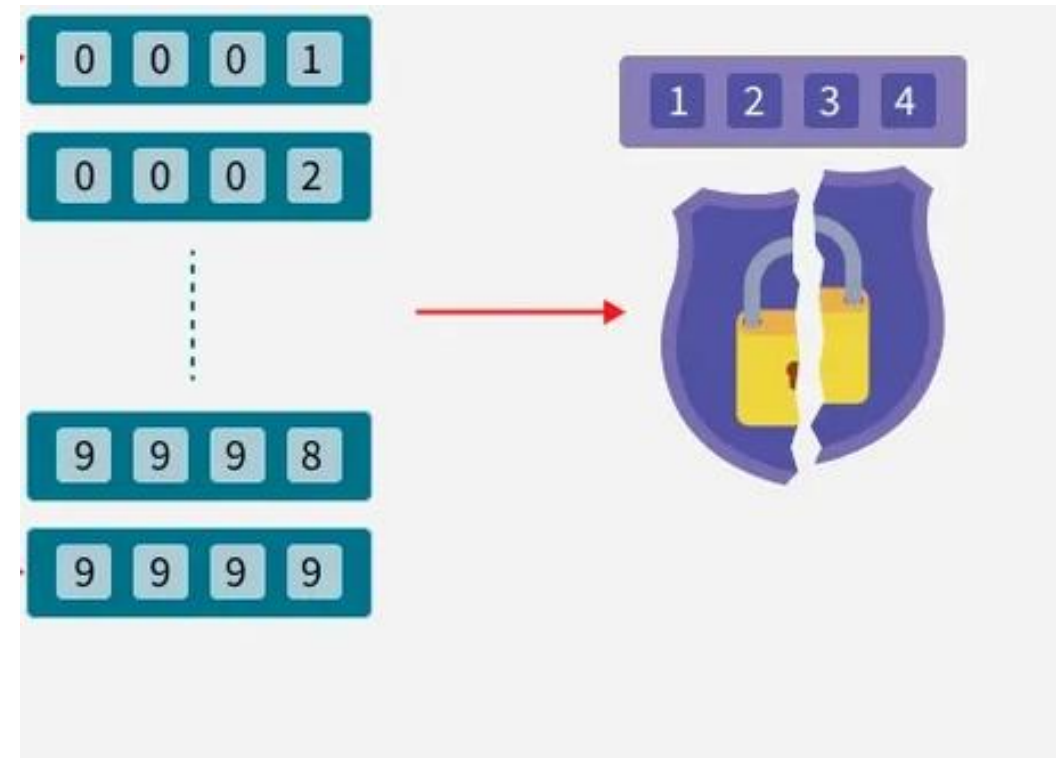
완전탐색 (Brute Force)



문제의 정답이 될 수 있는 모든 후보를 생성하고 검사함
조건에 맞는 답이 존재한다면 100% 정답을 보장함

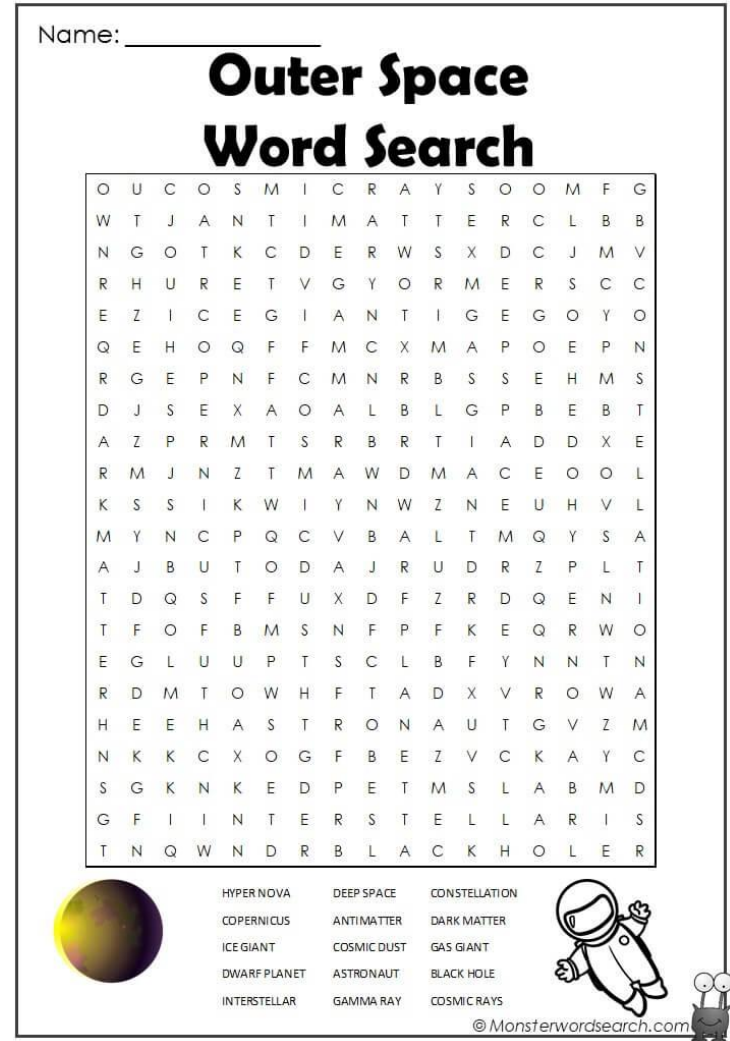
완전탐색 == 나쁜 알고리즘?

- 입력의 크기가 충분히 작다면 구현이 가장 쉬우면서 정답률 100%인 최선의 알고리즘
- 복잡한 알고리즘 설계 전 베이스로 활용됨
- 브루트포스가 불가능한 문제는 DP, Greedy와 같은 더욱 효율적인 알고리즘도 불가능함



Search Space

- 문제에서 고려해야 하는 가능한 모든 후보를 의미함
- 1~N 까지의 자연수 중 짝수를 모두 찾으시오.
여기서 Search Space는 1~N 까지의 자연수임
그 중 $N \% 2 == 0$ 을 만족하는 후보가 정답



이거 전체가 Search Space임
이 중 맞는 단어가 정답이 됨

Permutation: 순열

Permutation

$${}^n P_r = \frac{n!}{(n-r)!}$$

where,

- n = Number of Total Objects
- r = Number of Objects Chosen at Once
- $0 \leq r \leq n$

서로 다른 n 개 중 r 개를 선택한 다음 순서를 고려하여 나열하는 경우의 수

Permutation: 순열

```
arr = [1, 2, 3]

for i in range(len(arr)):
    for j in range(len(arr)):
        for k in range(len(arr)):
            if i != j and j != k and i != k:
                print(arr[i], arr[j], arr[k])
```

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

중첩 반복문으로 구현 가능 (3P3)

Combination: 조합

Combinations

$${}^nC_r = \frac{n!}{r!(n-r)!}$$

where,

- n = Number of Total Objects
- r = Number of Objects Chosen at Once
- $0 \leq r \leq n$

서로 다른 n 개 중 r 개를 선택한 다음 순서를 고려하지 않고 나열하는 경우의 수

Combination: 조합

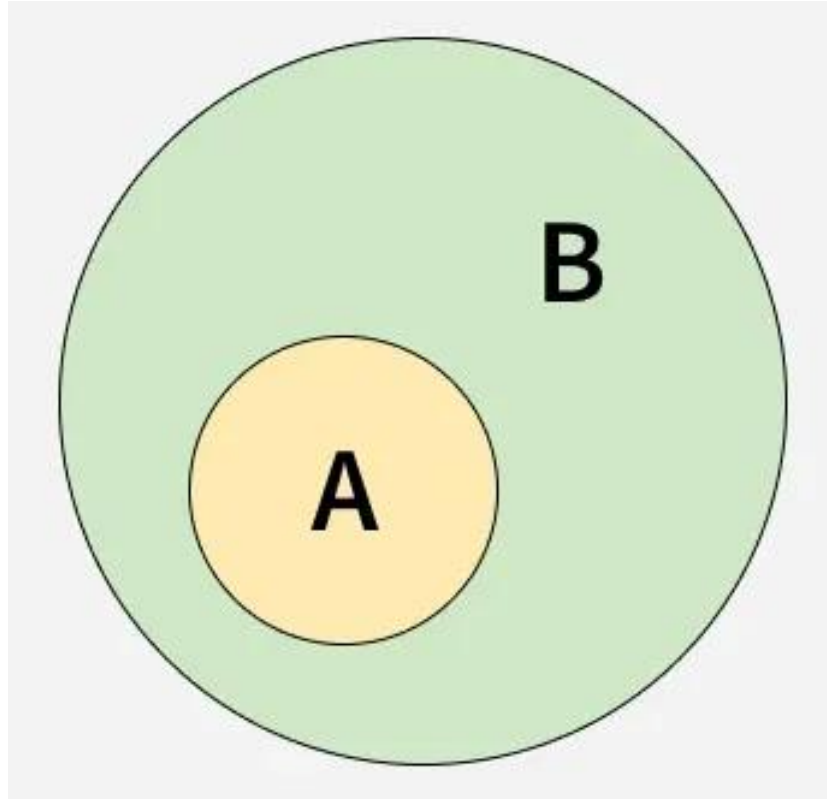
```
arr = [1, 2, 3, 4]

for i in range(len(arr)):
    for j in range(i + 1, len(arr)):
        for k in range(j + 1, len(arr)):
            print(arr[i], arr[j], arr[k])
```

```
1 2 3
1 2 4
1 3 4
2 3 4
```

중첩 반복문으로 구현 가능 (4C3)

Subset / Power Set: 부분집합



N개의 원소 각각에 대해 선택한다 / 선택하지 않는다 결정

Subset / Power Set: 부분집합

```
arr = [1, 2, 3]

for i in range(2):
    for j in range(2):
        for k in range(2):
            subset = []

            if i == 1:
                subset.append(arr[0])
            if j == 1:
                subset.append(arr[1])
            if k == 1:
                subset.append(arr[2])

            print(subset)
```

```
[]
[3]
[2]
[2, 3]
[1]
[1, 3]
[1, 2]
[1, 2, 3]
```

완전탐색 체크리스트

1. 무엇을 선택해야 하는가?
2. 순서가 중요한가?
3. 전체 탐색 공간의 크기는 얼마인가?
4. 제한시간 내에 모든 경우를 탐색할 수 있는가?

완전탐색은 맨 처음에 모든 경우가 무엇인지 정의하는게 핵심임

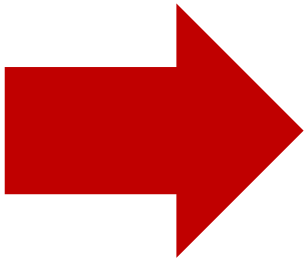
02

백트래킹

Backtracking

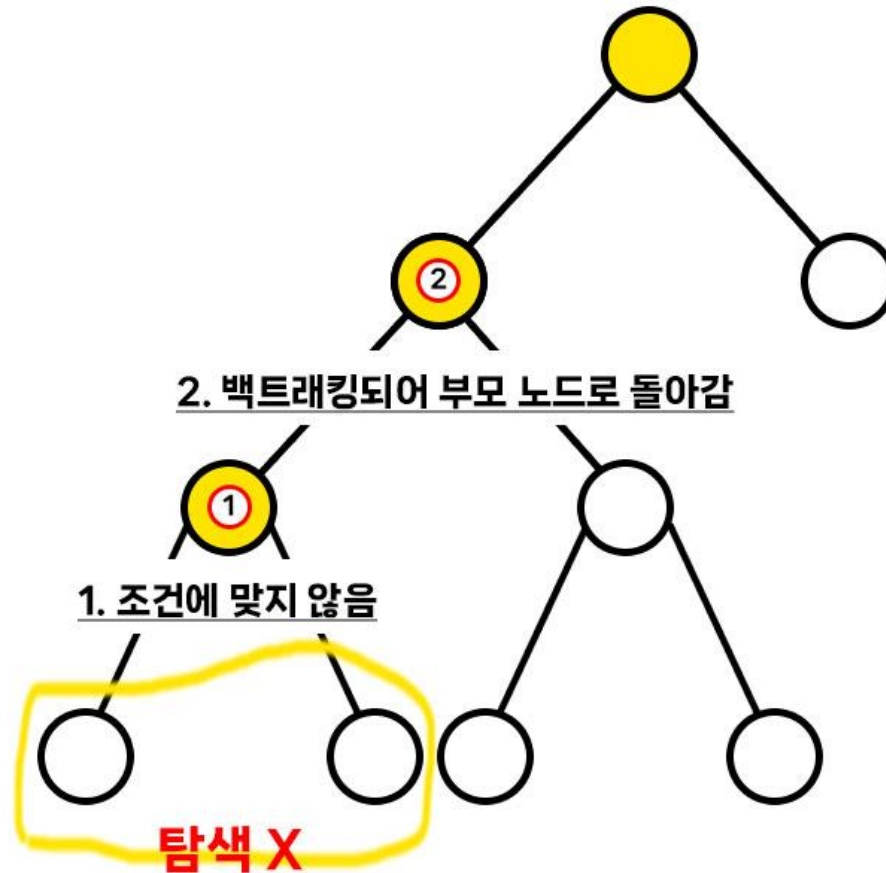
완전탐색의 단점

- 15개의 숫자를 순서를 고려하여 일렬로 늘어났을 때 특정한 값이 있는지 보는 상황
- 순열로 표현하면 15P15임
- 이는 15!, 약 1.3조임
- 초당 1억번 연산 시 약 3.63시간 소요



답이 될 수 없는 길은 미리 잘라내면?

백트래킹

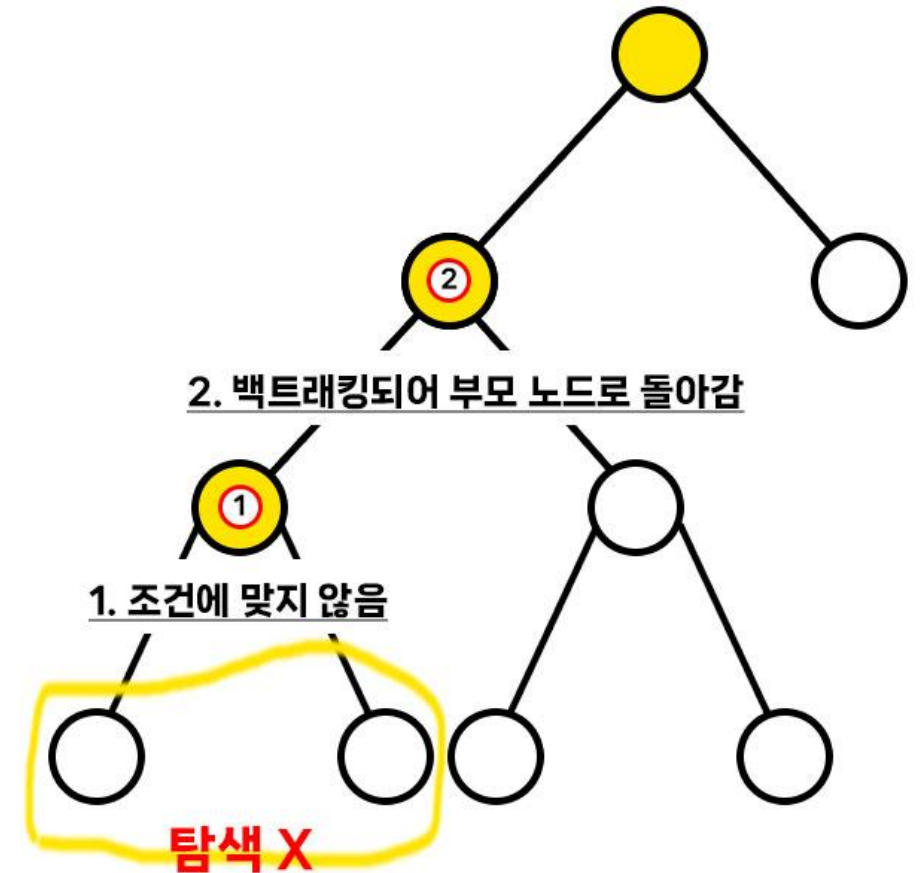


답이 될 가능성이 없는 길은 미리 잘라냄

막다른 길에 도달하면 이전 갈림길로 돌아가 다른 길 탐색

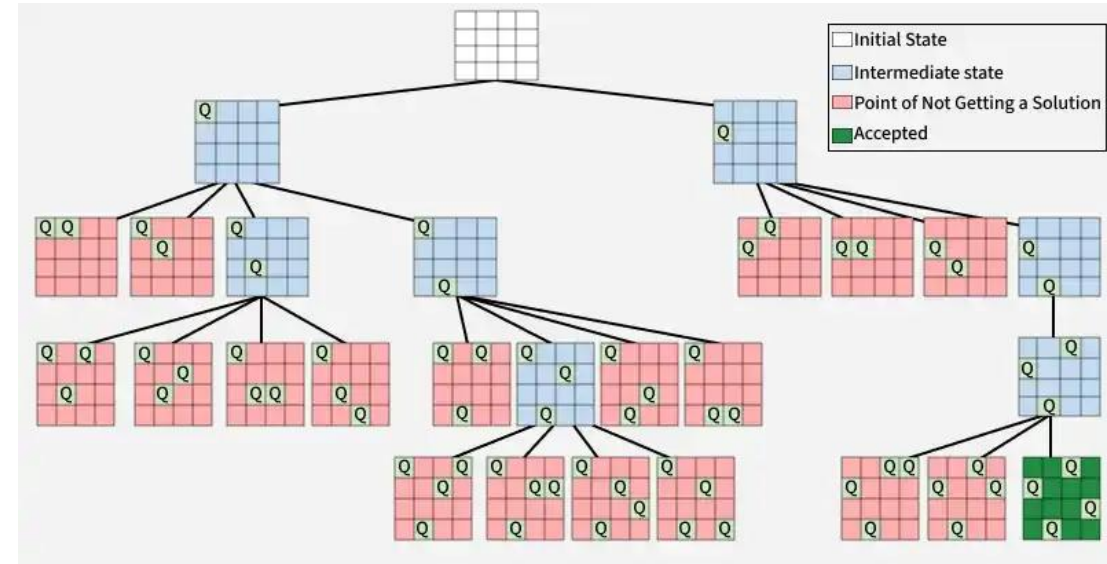
백트래킹

- Promising: 진행중인 경로가 해답을 찾을 가능성이 있을 때
- Non-promising: 진행중인 경로가 해답을 찾을 가능성이 0%일 때
- Pruning: Promising하지 않은 하위 트리를 잘라냄
- 주로 DFS로 구현되고, DFS는 주로 스택 또는 재귀함수로 구현됨

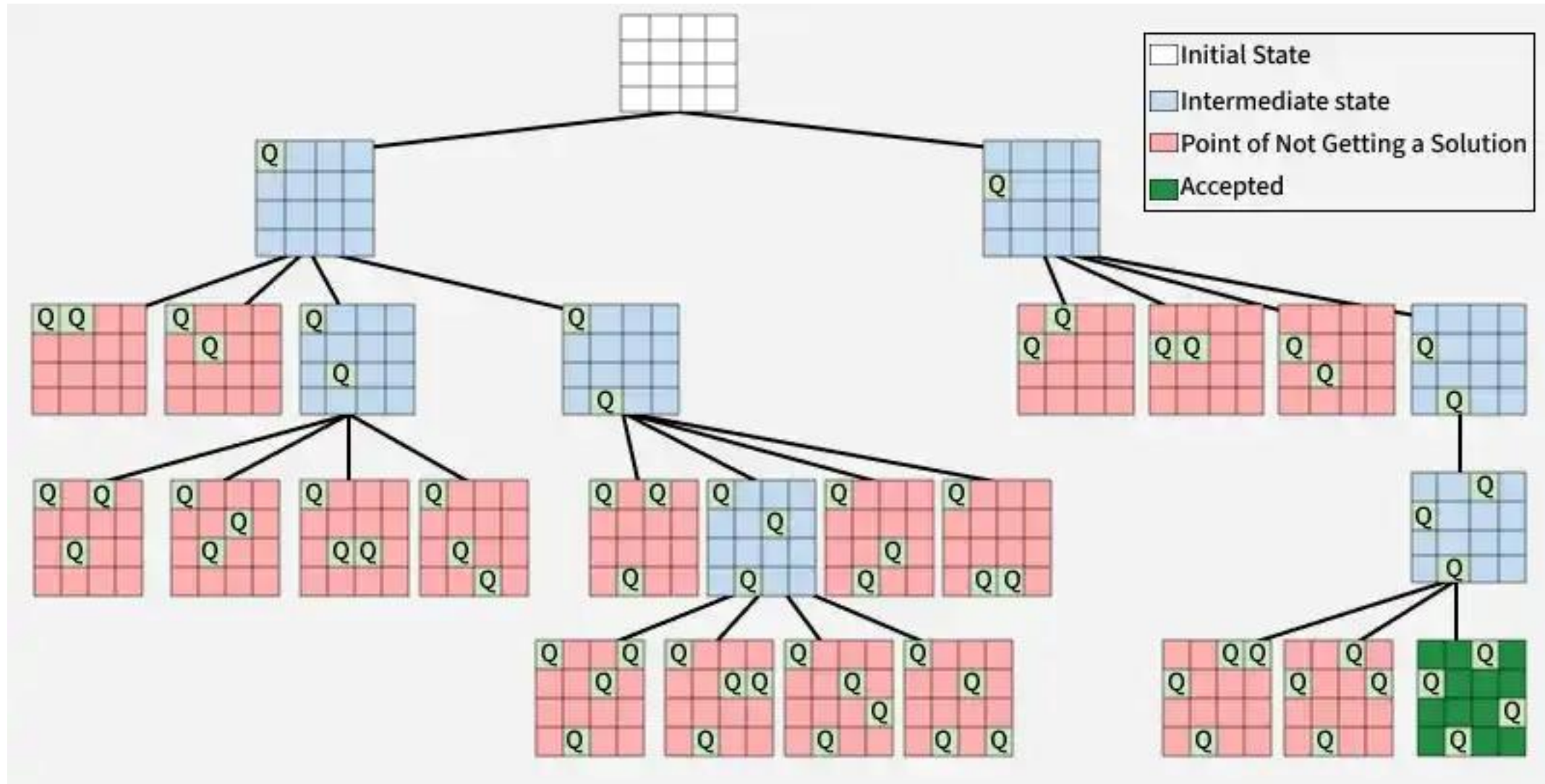


백트래킹: N-Queen Problem

1. 퀸을 행에 하나씩 놓기 (위에서부터)
2. 가로세로 확인
3. 불가능한 배치면 백트래킹
4. 답을 찾을 때까지 반복



백트래킹: N-Queen Problem



백트래킹 체크리스트

1. 현재 상태에서 무엇을 선택할 수 있는가?
2. 선택한 뒤 무엇이 변하는가?
3. 언제 하나의 탐색이 끝나는가?
4. 더 탐색할 필요가 없는 경우
(Pruning의 조건)는 무엇인가?

03

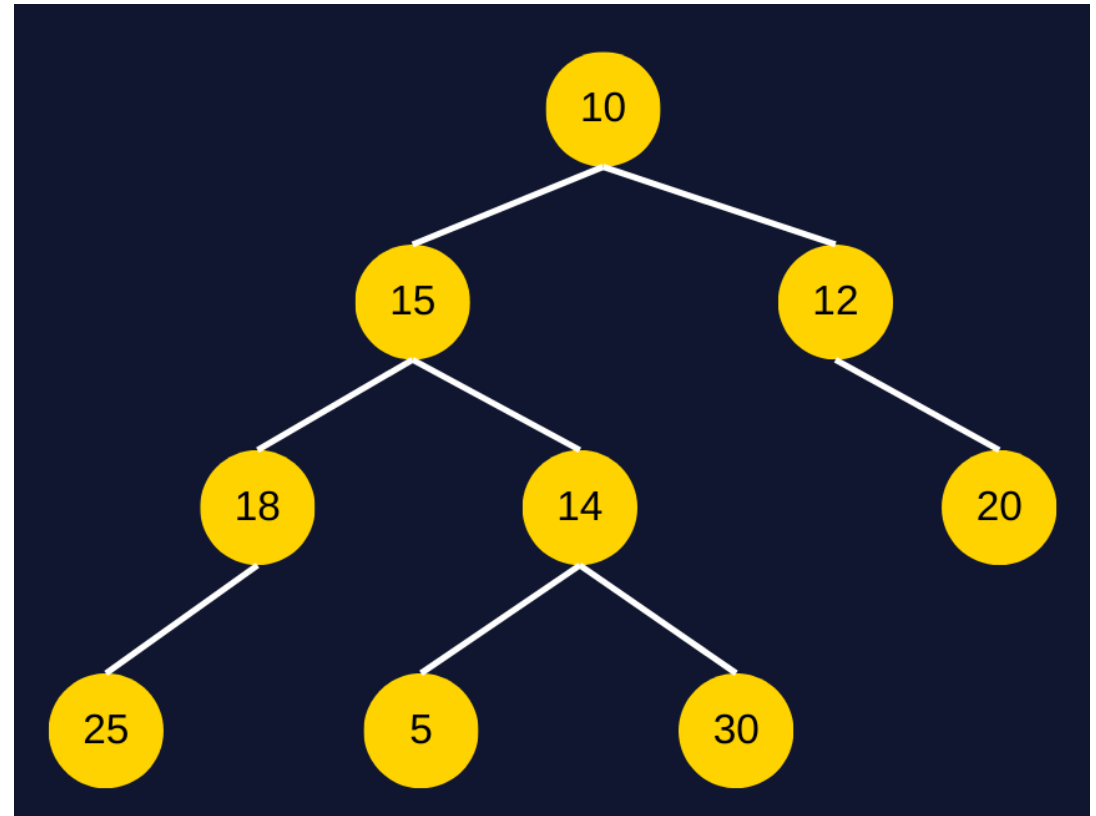
Greedy

Greedy Algorithm

Greedy Algorithm

- 매 탐색 단계에서 항상 최선의 경우를 선택
- 한번 내린 선택을 되돌리지 않음
- 빠르고 구현이 상당히 단순함

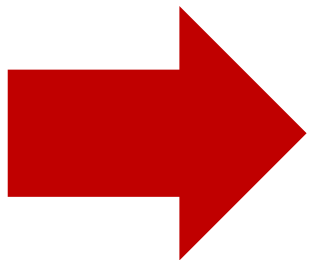
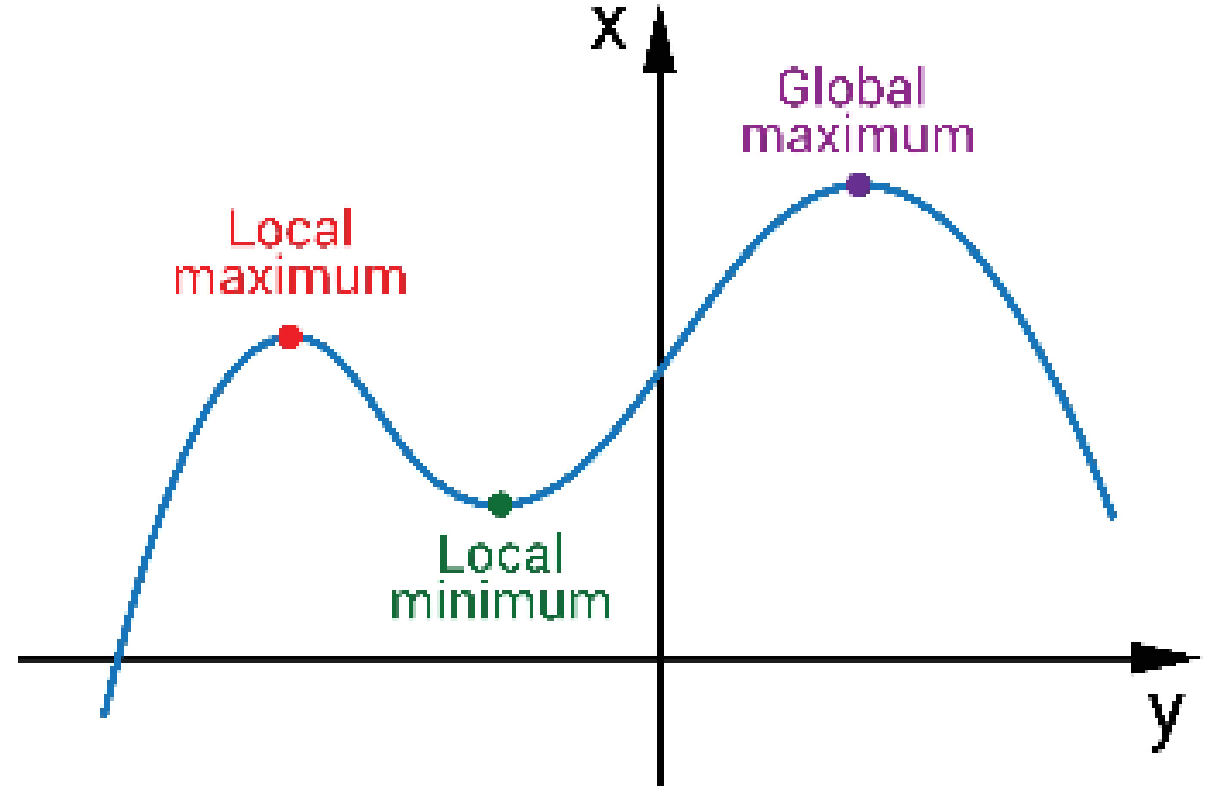
그런데 지금 가장 좋은 선택을 하면
최종 결과도 가장 좋을까?



Greedy의 탐색 순서는
10 → 15 → 18 → 25 임

Optimum

- **Local Optimum**
현재 선택 단계에서 눈 앞에 보이는
가장 좋은 선택
- **Global Optimum**
전체 문제에서 얻을 수 있는
가장 뛰어난 정답



매번 Local Optimum을 선택한다고
Global Optimum이 보장되는건 아님

Greedy가 정답을 보장할 수 있는 조건

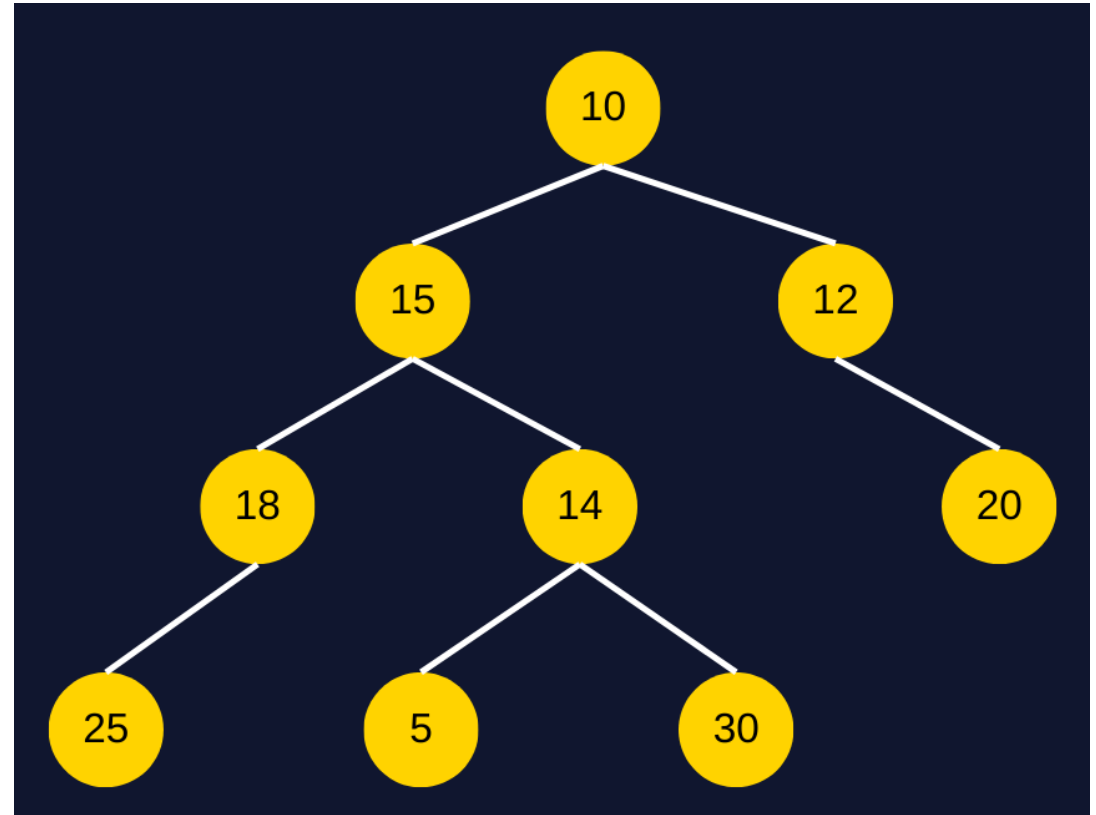
- **Greedy Choice Property**

앞의 Greedy한 선택이 이후의 선택에 악영향을 주지 않을 때

- **Optimal Substructure**

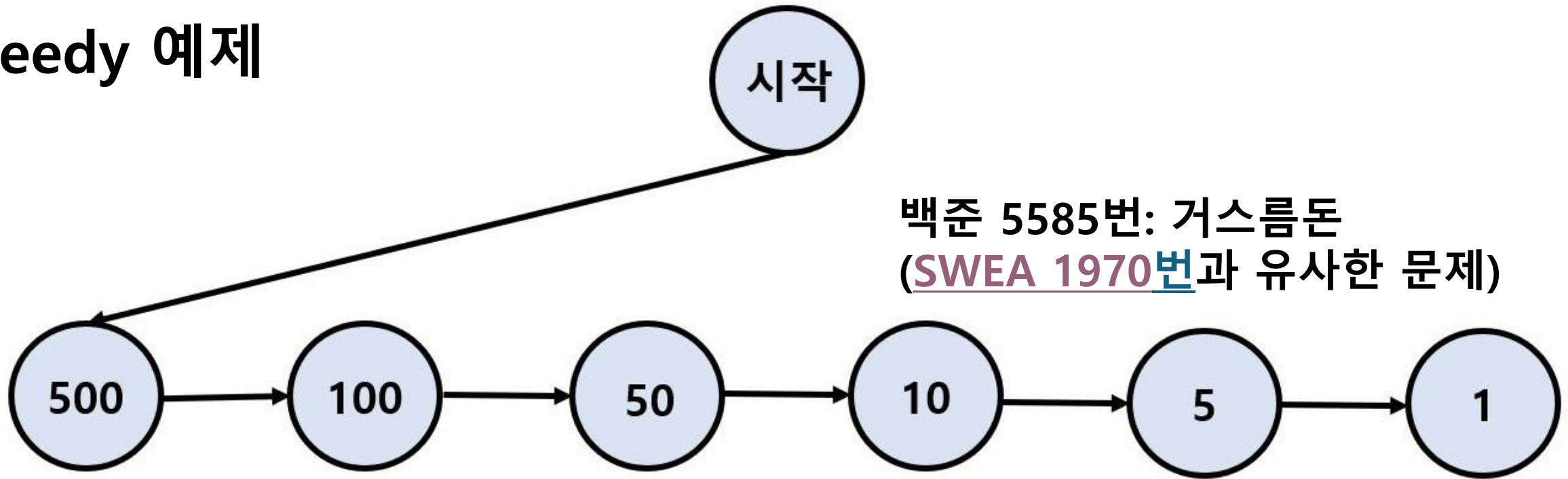
부분 문제의 최적해를 모으면
전체 문제의 최적해가 될 때

- 위의 두 성질이 증명되지 않을 때
Greedy를 쓰면 정답을 보장할 수 없음



트리를 다 내려갔을 때
전체의 최적해는 30이지만
Greedy는 25가 나옴

Greedy 예제

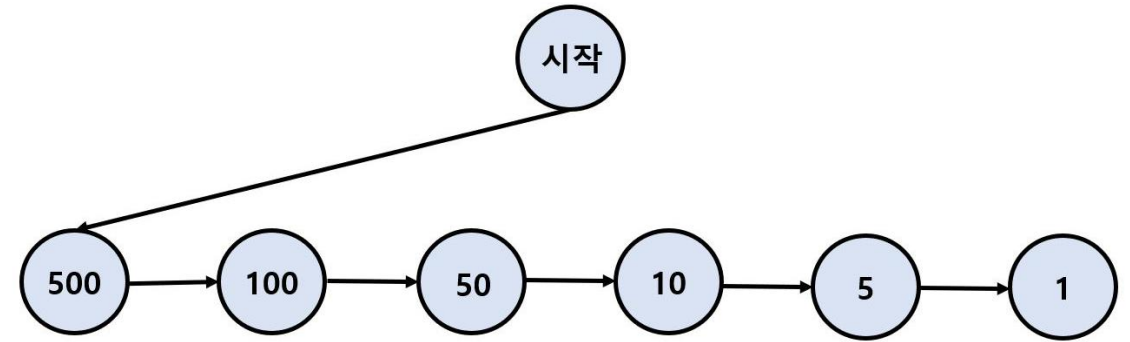


백준 5585번: 거스름돈
([SWEA 1970번](#)과 유사한 문제)

- 잡화점에서 물건을 사고 거스름돈을 받음
- 잡화점은 500, 100, 50, 10, 5, 1원짜리 동전이 충분히 있음
- 1000원을 내고 거스름돈을 받을 때, 동전의 개수를 최소로 하려면?

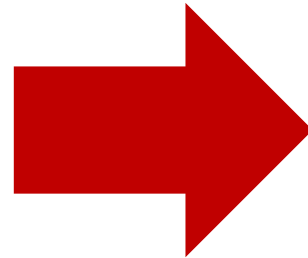
Greedy 예제

총 거스름돈: 876원



1. 가장 액수가 큰 동전부터 최대한 많이 선택 (Greedy 하게)

2. 500원: 1개, 남은 금액 376원
3. 100원: 3개, 남은 금액 76원
4. 50원: 1개, 남은 금액 26원
5. 10원: 2개, 남은 금액 6원
6. 5원: 1개, 남은 금액 1원
7. 1원: 1개, 남은 금액 0원



최종 동전 개수: 9개
최적 해 보장

Greedy 예제: Greedy를 써도 되는 이유

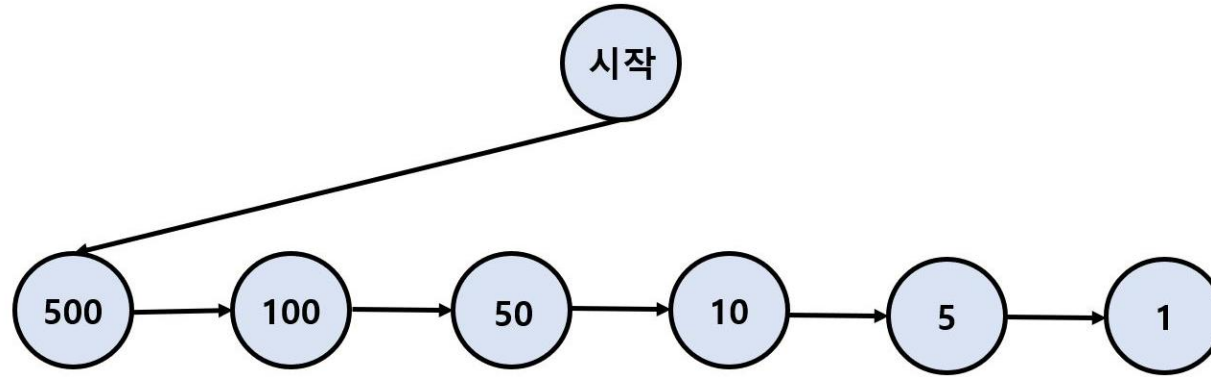
- Greedy Choice Property: 현재 사용할 수 있는 가장 큰 동전을 최대한 선택해도 이후의 최적해를 망치지 않음

예시: 1000원은 500원 2개, 100원 10개, ... 1원 1000개로 만들 수 있음
큰 동전을 작은 동전으로 바꿨을 때 필요한 개수가 줄어드는 경우가 없음

- Optimal Substructure: 현재 최적 선택을 하고 남은 금액의 최적해를 구하는 동일한 구조의 부분 문제로 만들 수 있음

예시: 거스름돈 850원의 최적해 $\rightarrow$ 350원의 최적해 $\rightarrow$ 50원의 최적해 $\rightarrow$ 0

Greedy 예제: Greedy를 쓰면 안될 때



- 만약 동전이 400, 300, 100원 짜리이고, 거스름돈이 600원이라면?
- Greedy: $400 + 100 + 100$: 동전 3개
최적 해: $300 + 300$: 동전 2개
- 큰 액수의 동전이 작은 액수의 동전의 배수가 아니라서 Greedy Choice Property가 성립하지 않음
(앞선 Greedy한 선택이 이후의 선택에 악영향을 줌)

Greedy Algorithm 체크리스트

1. 매 순간 가장 좋아 보이는 선택은 무엇인가?
2. 지금 최선의 선택을 해도 전체 최적해를 구할 수 있는가?
3. 현재 선택 후의 남은 문제도 최적으로 풀면 전체도 최적이 되는가?
4. 해당 문제에 반례가 존재하는가?